REACT

Conceptos Esenciales para Entrevista

Basado en el proyecto Around the U.S. — TripleTen Sprint 11 & 12

Stack: React 19 · TypeScript · Vite · Context API · REST API
Repositorio: web_project_around_react

1. ¿Qué es React y por qué usarlo?

React es una **biblioteca de JavaScript/TypeScript** para construir interfaces de usuario. Su idea central: la UI es una **función del estado**. Cuando el estado cambia, React recalcula qué parte del DOM necesita actualizarse y solo toca eso (Virtual DOM diff).

Antes (proyecto vanilla Around the U.S.) manipulabas el DOM directamente: `document.querySelector('.profile__title').textContent = data.name`. Con React declaras el resultado esperado y React hace el trabajo:

```
// React — declarativo
<h1 className='profile__title'>{currentUser?.name}</h1>

// Vanilla JS — imperativo
document.querySelector('.profile__title').textContent = userData.name;
```

- Sin **querySelector**, **getElementById** ni manipulación manual del DOM.
- Las actualizaciones son automáticas cuando el estado cambia.
- El código es más predecible y testeable.

2. Componentes Funcionales y JSX/TSX

Un **componente funcional** es una función TypeScript que recibe props y devuelve JSX. En tu proyecto, cada parte visual es un componente propio:

```
// src/components/Footer/Footer.tsx
export default function Footer(): React.JSX.Element {
  return (
    <footer className='footer page__section'>
      <p className='footer__copyright'>© 2025 Around The U.S.</p>
    </footer>
  );
}
```

Reglas de JSX que seguiste

- El marcado siempre dentro de **paréntesis** en el return.
- **className** en lugar de class (es atributo JS, no HTML).
- Un solo elemento raíz por componente (o usar <>… Fragment).
- Expresiones JS dentro de **{ }**: {currentUser?.name}.
- Eventos en camelCase: **onClick**, **onSubmit**, **onChange**.

Árbol de componentes del proyecto

```
App
■■■ Header
■■■ Main
■   ■■■ Card (× n — una por tarjeta de la API)
■   ■■■ Popup (genérico)
■       ■■■ EditProfile
■       ■■■ EditAvatar
■       ■■■ NewCard
■       ■■■ RemoveCard
■       ■■■ ImagePopup
■■■ Footer
```

■ *Pregunta típica: '¿Cuándo conviene crear un componente?' Cuando una parte de la UI se repite, tiene su propia lógica o necesita ser reutilizable. Card es un ejemplo: se renderiza una vez por cada objeto del array cards.*

---

3. Props — Comunicación padre → hijo

Las **props** son los argumentos de un componente: datos y funciones que el padre pasa al hijo. Son de solo lectura; el hijo no las modifica.

```
// Main.tsx — pasa props a Card
<Card
  key={card._id}
  card={card}
  onImageClick={handleCardImageClick}
  handleCardLike={handleCardLike}
  handleCardDeleteClick={handleCardDeleteClick}
/>

// Card.tsx — recibe y usa las props
type CardProps = {
  card: CardData;
  onImageClick: (card: CardData) => void;
  handleCardLike: (card: CardData) => void;
  handleCardDeleteClick: (card: CardData) => void;
};

export default function Card(props: CardProps): React.JSX.Element {
  const { card, onImageClick, handleCardLike, handleCardDeleteClick } = props;
  // ...
}
```

prop especial: key

Cuando renderizas una lista con `.map()`, React necesita `key` para identificar cada elemento y hacer el diff eficientemente. Usa siempre el ID real del dato, nunca el índice del array.

```
// Correcto — ID único del objeto
{cards.map((card) => (
  <Card key={card._id} card={card} ... />
))}

// Incorrecto — índice como key
{cards.map((card, index) => (
  <Card key={index} card={card} ... />  // ■■ evitar
))}
```

---

4. Estado con useState

useState guarda datos que, al cambiar, hacen que React vuelva a renderizar el componente. La función de actualización **reemplaza** el valor; no lo muta.

```
// App.tsx — tres variables de estado
const [currentUser, setCurrentUser] = useState<UserData | null>(null);
const [cards, setCards] = useState<CardData[]>([]);
const [popup, setPopup] = useState<ModalData | null>(null);
```

Actualización inmutable de arrays

Nunca uses `push()` ni mutación directa. React compara referencias; si mutas el array, no detecta el cambio.

```
// Like: reemplazar una tarjeta dentro del array (map devuelve uno nuevo)
setCards((state) =>
  state.map((c) => (c._id === card._id ? newCard : c))
);

// Nueva tarjeta: prepend al inicio del array (spread)
setCards((state) => [newCard, ...state]);

// Borrar: filtrar fuera la tarjeta eliminada
setCards((state) => state.filter((c) => c._id !== card._id));
```

Estado levantado (Lifting State Up)

El Sprint 12 movió `cards`, `currentUser` y `popup` de `Main` a `App`. Regla: el estado vive en el ancestro común más cercano de los componentes que lo necesitan.

> ■ *Pregunta típica: '¿Por qué no poner useState en cada componente?' Porque si dos componentes necesitan el mismo dato, deben leerlo del mismo origen. El lifting garantiza una sola fuente de verdad.*

---

5. Efectos con useEffect

useEffect ejecuta código que tiene efectos secundarios (llamadas a APIs, suscripciones, timers) fuera del renderizado.

```
// App.tsx — carga usuario y tarjetas UNA sola vez al montar
useEffect(() => {
  (async () => {
    try {
      const [userData, initialCards] = await Promise.all([
        api.getUserInfo(),
        api.getInitialCards(),
      ]);
      setCurrentUser(userData);
```

```
      setCards(initialCards);
    } catch (error) {
      console.error(error);
    }
  })();
}, []); // <-- array vacío = solo se ejecuta al montar
```

- **[ ]** (array vacío): se ejecuta una vez al montar el componente.
- **[dep1, dep2]**: se ejecuta cada vez que dep1 o dep2 cambian.
- **Sin array**: se ejecuta después de cada renderizado (raramente útil).

Promise.all lanza las dos peticiones en paralelo. Si fueran secuenciales (await una, await la otra), el tiempo total sería la suma de ambas.

---

6. Context API — Estado global sin prop drilling

Prop drilling: pasar props a través de componentes intermedios que no los usan. Context resuelve esto compartiendo datos directamente a cualquier nivel del árbol.

Los tres pasos de Context en tu proyecto

```
// PASO 1 — Crear el contexto (CurrentUserContext.tsx)
import { createContext } from 'react';
import type { CurrentUserContextType } from '../interfaces/CurrentUserContextType';

const CurrentUserContext = createContext<CurrentUserContextType>(
  {} as CurrentUserContextType,
);
export default CurrentUserContext;

// PASO 2 — Proveer valores (App.tsx)
<CurrentUserContext.Provider
  value={{
    currentUser,
    handleUpdateUser,
    handleUpdateAvatar,
    handleAddPlaceSubmit,
    handleCardDelete,
  }}
>
  <div className='page__content'>
    <Header />
    <Main ... />
    <Footer />
  </div>
</CurrentUserContext.Provider>

// PASO 3 — Consumir en cualquier componente (EditProfile.tsx)
const { currentUser, handleUpdateUser } = useContext(CurrentUserContext);
```

¿Qué va en el contexto de tu app?

- `currentUser`: datos del usuario logueado (nombre, avatar, _id).
- `handleUpdateUser`: actualiza perfil → llama API → setCurrentUser → cierra popup.
- `handleUpdateAvatar`: igual pero para el avatar.
- `handleAddPlaceSubmit`: crea tarjeta → prepend al array.
- `handleCardDelete`: borra tarjeta → filtra el array.

■ *Pregunta típica: '¿Context reemplaza a Redux?' No necesariamente. Context es ideal para datos globales de bajo cambio (usuario, tema). Redux o Zustand se usan para estado complejo con muchas actualizaciones.*

---

7. Formularios: Controlados vs No controlados

Componente controlado — EditProfile.tsx

React **controla** el valor del input: cada tecla dispara onChange, que actualiza el estado, que actualiza el value del input. Es la fuente de verdad.

```
// EditProfile.tsx
const [name, setName] = useState(currentUser?.name || '');

<input
  type='text'
  value={name}                          // ← React controla el valor
  onChange={(e) => setName(e.target.value)}  // ← actualiza el estado
/>
```

✔ Ventaja: validación en tiempo real, deshabilitación condicional del botón.

Componente no controlado — EditAvatar.tsx

useRef guarda una referencia al nodo DOM sin disparar rerenders. Solo lees el valor cuando el usuario hace submit.

```
// EditAvatar.tsx
const avatarRef = useRef<HTMLInputElement>(null);

function handleSubmit(event: React.FormEvent<HTMLFormElement>): void {
  event.preventDefault();
  if (avatarRef.current) {
    void handleUpdateAvatar({ avatar: avatarRef.current.value });
  }
}

<input type='url' ref={avatarRef} required />
```

✔ Ventaja: más simple cuando no necesitas validar en tiempo real.

Nueva tarjeta — NewCard.tsx (controlado, estado limpio)

El componente se monta de nuevo cada vez que se abre el popup. Inicializar con '' garantiza campos vacíos sin resetear manualmente.

```
const [name, setName] = useState('');  // siempre vacío al montar
const [link, setLink] = useState('');
```

---

8. Comunicación con la API — Clase Api

Toda la comunicación con el servidor vive en `src/utils/api.ts`. Se exporta una instancia ya configurada, no la clase.

```
// utils/api.ts (fragmento)
class Api {
  private _baseUrl: string;
  private _headers: Record<string, string>;
```

```
  private async _request<T>(url: string, options?: RequestInit): Promise<T> {
    const res = await fetch(url, options);
    if (res.ok) return (await res.json()) as T;
    throw new Error(`Error: ${res.status}`);
  }

  async addLike(id: string): Promise<CardData> {
    return await this._request<CardData>(`${this._baseUrl}/cards/${id}/likes`, {
      method: 'PUT',
      headers: this._headers,
    });
  }

  async removeLike(id: string): Promise<CardData> {
    return await this._request<CardData>(`${this._baseUrl}/cards/${id}/likes`, {
      method: 'DELETE',
      headers: this._headers,
    });
  }
}

// Instancia lista para importar
const api = new Api({ baseUrl: '...', headers: { authorization: '...' } });
export default api;
```

Toggle de like — lógica optimista

```
// App.tsx
async function handleCardLike(card: CardData): Promise<void> {
  const isLiked = card.isLiked;
  try {
    const apiCall = isLiked ? api.removeLike(card._id) : api.addLike(card._id);
    const newCard = await apiCall;          // servidor devuelve la tarjeta actualizada
    setCards((state) =>
      state.map((c) => (c._id === card._id ? newCard : c))
    );
  } catch (error) {
    console.error(error);
  }
}
```

---

9. TypeScript en React — Interfaces y Tipos

TypeScript atrapa errores en tiempo de compilación, no en tiempo de ejecución.

Interfaces del proyecto (src/interfaces/)

```
// CardData.ts
export interface CardData {
  _id: string;
  name: string;
  link: string;
  owner: string;   // string, no objeto — coincide exactamente con la API
  createdAt: string;
  isLiked: boolean;
}

// UserData.ts
```

```
export interface UserData {
  _id: string;
  name: string;
  about: string;
  avatar: string;
}

// CurrentUserContextType.ts
export interface CurrentUserContextType {
  currentUser: UserData | null;
  handleUpdateUser: (data: ProfileFormData) => Promise<void>;
  handleUpdateAvatar: (data: AvatarFormData) => Promise<void>;
  handleAddPlaceSubmit: (data: CardFormData) => Promise<void>;
  handleCardDelete: (card: CardData) => Promise<void>;
}
```

Genéricos en la clase Api

```
// _request<T> — T es el tipo que devuelve la petición
private async _request<T>(url: string, options?: RequestInit): Promise<T> {
  const res = await fetch(url, options);
  if (res.ok) return (await res.json()) as T;
  throw new Error(`Error: ${res.status}`);
}

// Uso: TypeScript sabe que esto devuelve CardData[]
async getInitialCards(): Promise<CardData[]> {
  return await this._request<CardData[]>(`${this._baseUrl}/cards/`, { ... });
}
```

■ *Pregunta típica: '¿Por qué owner es string y no un objeto User?' Porque la API real devuelve solo el _id del dueño como string. Si defines owner como UserData, TypeScript falla al parsear la respuesta.*

---

10. Reglas de los Hooks

- **Solo en componentes funcionales o en custom hooks** — nunca en clases.
- **Solo al nivel superior** — nunca dentro de if, for, o funciones anidadas.
- **Mismo orden en cada render** — React los identifica por posición.

```
// ■ Correcto
export default function EditProfile() {
  const { currentUser } = useContext(CurrentUserContext);  // nivel superior
  const [name, setName] = useState(currentUser?.name || '');

  if (!currentUser) return null;  // ← condicional DESPUÉS de los hooks
}

// ■ Incorrecto
export default function EditProfile() {
  if (!currentUser) return null;
  const [name, setName] = useState('');  // ← hook dentro de condicional
```

---

11. Flujo completo de la aplicación

Traza de lo que ocurre cuando el usuario abre la app y borra una tarjeta:

1. `main.tsx → monta <App /> en el DOM`

```
2. App.tsx — useEffect([], [])
   → Promise.all([api.getUserInfo(), api.getInitialCards()])
   → setCurrentUser(userData)   → re-render con perfil real
   → setCards(initialCards)     → re-render con tarjetas de la API

3. Main recibe cards como prop → renderiza un <Card /> por cada tarjeta

4. Card → compara owner === currentUser._id
   → solo muestra el botón de borrar en tarjetas propias

5. Usuario click "borrar" → handleCardDeleteClick(card)
   → Main abre popup RemoveCard con la tarjeta seleccionada

6. Usuario click "Sí" en RemoveCard
   → handleCardDelete(card) del contexto
   → api.deleteCard(card._id) — petición DELETE al servidor
   → setCards(state.filter(...)) — elimina del array local
   → setPopup(null) — cierra el popup
```

---

12. Preguntas frecuentes de entrevista

P: ¿Qué es el Virtual DOM?

R: Una copia en memoria del DOM real. React compara el Virtual DOM anterior con el nuevo (proceso llamado 'reconciliation' o 'diffing') y solo actualiza los nodos que cambiaron. Esto evita repintados completos del navegador.

P: ¿Diferencia entre props y state?

R: Props: datos que vienen del padre, de solo lectura dentro del hijo. State: datos internos del componente que, al cambiar, disparan un re-render.

P: ¿Cuándo usar useContext vs props?

R: Props para datos locales entre padre e hijo directo. Context cuando muchos componentes en diferentes niveles necesitan el mismo dato (currentUser, tema, idioma).

P: ¿Qué pasa si muto el estado directamente?

R: React no detecta el cambio porque compara referencias. setCards([...cards, newCard]) crea un array nuevo; cards.push(newCard) muta el existente y React no re-renderiza.

P: ¿Para qué sirve el array de dependencias de useEffect?

R: Controla cuándo se vuelve a ejecutar el efecto. [] = solo al montar, [userId] = cada vez que userId cambie, sin array = en cada render.

P: ¿Diferencia entre componente controlado y no controlado?

R: Controlado: React es la fuente de verdad del valor del input (value + onChange). No controlado: el DOM guarda el valor y se lee con useRef cuando se necesita.

P: ¿Por qué key en las listas?

R: React usa key para identificar elementos entre renders. Sin key (o con el índice), React puede reutilizar el nodo del componente equivocado al reordenar o filtrar, generando bugs de estado.

P: ¿Qué hace async/await en los handlers?

R: Permite esperar respuestas de la API de forma legible. handleCardLike es async porque necesita esperar api.addLike() antes de actualizar el estado. void handleCardLike(card) en el onClick descarta la Promise (requerido por TypeScript/lint).